



PREDICTIVE MAINTENANCE

EPISODE 3

INDICATEURS DE PERFORMANCE
OUTILS D'ANALYSE DES DONNÉES

EMMANUEL GBRA

28 AVRIL 2026



Comme vu précédemment, nous avons abordé les KPI et leur mode de calcul, mais concrètement une autre question se pose, à savoir quels sont les outils qui permettent de concevoir ces KPI dans un contexte de maintenance prédictive. En réalité, les KPI ne sortent pas directement d'une simple observation, ils sont construits à partir de la donnée, et cette donnée doit d'abord être collectée, ensuite traitée, puis transformée en indicateurs exploitables. C'est donc tout un processus qui relève du domaine de la data, car sans données bien organisées il est impossible d'obtenir des KPI fiables et utiles pour la prise de décision.





1. Outils de collectes de données

Ces outils permettent de récupérer les informations nécessaires au suivi des équipements, qu'il s'agisse de l'historique des pannes, des interventions de maintenance, des temps d'arrêt ou encore des mesures provenant des capteurs en temps réel. On distingue ici deux grandes familles de données qui ne se collectent pas de la même manière et ne mobilisent pas les mêmes outils.

Les données classiques sont celles qui sont déjà stockées dans des bases de données ou dans des fichiers structurés, notamment dans les GMAO. Elles servent principalement à retrouver l'historique d'un équipement, les ordres de travail ou les rapports d'intervention. Les outils les plus utilisés pour ce type de données sont les bases de données relationnelles comme MySQL ou PostgreSQL qui fonctionnent avec le langage SQL, ainsi que des tableurs comme Excel ou LibreOffice Calc qui restent très présents dans les environnements industriels peu digitalisés.



Les données dynamiques, quant à elles, viennent directement des machines et des capteurs et arrivent en continu. Elles ne peuvent pas être collectées manuellement car elles sont générées à des fréquences très élevées.

Pour les récupérer, les signaux capteurs passent d'abord par un automate programmable (PLC) qui les lit via des protocoles terrain comme Modbus ou Profibus selon le constructeur, ensuite ces valeurs remontent vers un système de supervision SCADA ou DCS, et enfin elles sont archivées dans un Historian qui constitue la mémoire long terme de tous les signaux capteurs horodatés sur des mois ou des années.

Le plus répandu sur le marché est AVEVA Historian. Ces données sont ensuite extraites via des API spécifiques ou via des protocoles comme MQTT qui fonctionne selon une logique de publication-abonnement, c'est-à-dire qu'il publie les données selon des fréquences définies vers ses abonnés sans qu'ils aient besoin de faire une requête. Pour construire une chaîne de streaming à partir de ces données, on utilise des outils comme Apache Kafka ou Apache Spark Streaming qui permettent de transporter et traiter ces flux en continu vers le pipeline de traitement des données.



AVEVA
Historian

Dans des contextes pédagogiques ou dans des environnements peu digitalisés, il est également possible de reproduire ce même principe de collecte en temps réel de manière simplifiée en utilisant Excel avec l'extension Data Streamer, qui permet de récupérer des données directement depuis un port série connecté à un microcontrôleur comme Arduino.

Cette approche reproduit le principe de la chaîne industrielle complète, c'est-à-dire acquisition en temps réel, transmission et visualisation, dans un environnement accessible sans infrastructure complexe. C'est d'ailleurs l'approche que nous utiliserons dans notre exemple pratique.



2. Outils de traitement

Après la phase de collecte, on passe à une étape tout aussi importante qui est le traitement des données. En réalité, les données collectées ne sont presque jamais directement exploitables pour construire des KPI. Elles sont souvent désorganisées, incohérentes ou incomplètes, ce qui rend toute analyse fausse si on ne les prépare pas correctement.

Le traitement des données consiste donc à rendre ces informations homogènes et fiables. Concrètement, cela passe par plusieurs actions. Il faut d'abord uniformiser les formats, par exemple convertir toutes les valeurs numériques dans un même format ou s'assurer que les dates suivent toutes la même structure. Ensuite, il faut nettoyer les données en supprimant ou corrigeant les valeurs incohérentes, comme des valeurs négatives impossibles ou des pics anormaux qui peuvent fausser complètement un indicateur. Il faut également gérer les données manquantes, soit en les supprimant, soit en les remplaçant par des valeurs cohérentes selon le contexte, par exemple en utilisant la moyenne des valeurs voisines pour combler un trou dans une série temporelle. Une autre étape importante consiste à structurer les données dans le temps, car en maintenance prédictive la dimension temporelle est essentielle pour observer les dérives et les comportements des équipements.

Excel est souvent le premier réflexe car il permet de faire des corrections rapides, des filtres, des tris et des transformations simples directement sur les fichiers exportés depuis une GMAO ou un Historian. Cependant lorsque le volume de données devient important ou que les traitements deviennent plus complexes, Excel montre rapidement ses limites. Dans ce cas, Python devient l'outil de référence. Avec la librairie Pandas, il est possible de manipuler de grandes quantités de données, de filtrer automatiquement les anomalies, de transformer les formats et de regrouper les informations par période.

	A	B	C	D
1	Fruit	Jan	Feb	Mar
2	Apple	352	135	291
3	Blueberry	217	387	109
4	Blueberry	395	133	206
5	Grape	252	304	266
6	Grape	479	265	403
7	Watermelon	389	299	446
8	Apple	117	174	191
9	Blueberry	500	323	429
10	Blueberry	219	471	182
11	Blueberry	412	346	334
12	Watermelon	209	145	234
13	Apple	456	120	200
14	Blueberry	191	278	451
15	Blueberry	336	404	258
16	Apple	205	268	320
17	Apple	460	422	308
18	Grape	425	352	252
19	Watermelon	367	500	383
20	Watermelon	268	230	137

Split data into multiple sheets based on column value

	A	B	C	D	E	F
1	Fruit	Jan	Feb	Mar		
2	Apple	352	135	291		
3	Apple	117	174	191		
4	Apple	456	120	200		
5	Apple	205	268	320		
6	Apple	460	422	308		
7						
8						



La librairie NumPy vient compléter Pandas pour les calculs numériques sur les séries temporelles, comme le calcul de moyennes glissantes ou d'écarts-types sur des fenêtres de temps définies.



NumPy



Pour les données qui arrivent en flux continu, Apache Spark permet de réaliser ces mêmes traitements à grande échelle directement sur les flux sans avoir à les stocker au préalable, ce qui est particulièrement utile lorsqu'on veut traiter des données capteurs en temps réel.



Le traitement permet donc de passer de données brutes à des données fiables. Sans cette étape, les KPI calculés derrière n'ont aucune valeur car ils reposeraient sur des informations erronées.

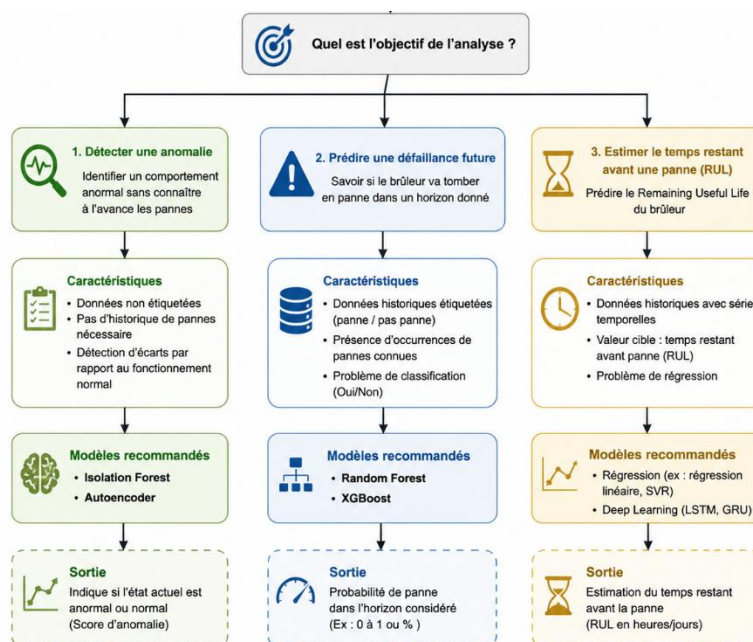
3. Outils de Transformations des données

Il est important de préciser que l'ensemble de ces étapes est généralement réalisé par des équipes de data engineers et de data scientists. Dans une approche de maintenance prédictive avancée, le chargé de maintenance doit également être capable de concevoir et de comprendre des pipelines de données, c'est-à-dire l'enchaînement structuré des étapes de collecte, transformation, analyse et restitution. Cette compétence devient stratégique car elle permet de mieux interpréter les résultats des modèles, de définir des besoins pertinents et d'assurer une meilleure intégration des solutions d'intelligence artificielle dans les systèmes industriels.

Une fois les données collectées et traitées, vient l'étape de leur transformation en KPI exploitables. Mais avant même de parler d'outils, cette transformation ne peut pas se faire sans réflexion préalable car construire un KPI sans se poser les bonnes questions revient à produire un indicateur qui ne sert à rien.

C'est pourquoi cette étape commence par la construction d'un arbre décisionnel qui va guider toute la démarche en répondant à six questions fondamentales

- Quelle est la cible que je veux surveiller, c'est-à-dire quel équipement, quel composant ou quel processus est concerné ?
- Quel comportement est-ce que je cherche à détecter, une dérive progressive, une panne soudaine ou une dégradation lente ?
- Quelle donnée me permet d'observer ce comportement et où est-elle disponible, dans ma GMAO, dans mon Historian ou dans mes fichiers Excel ?
- À quelle fréquence cette donnée est-elle générée et à quelle fréquence mon KPI doit-il être mis à jour ?
- Quel calcul dois-je appliquer pour transformer cette donnée brute en indicateur significatif ?
- Qui va consommer ce KPI et dans quel outil doit-il être affiché ?





Une fois cet arbre construit, on sait exactement ce qu'on veut produire comme KPI, où se trouve la donnée source, à quelle fréquence elle doit être traitée et sous quelle forme le résultat doit être présenté.

C'est également à ce moment que se pose la question du modèle prédictif à utiliser car en maintenance prédictive le KPI ne se calcule pas toujours par une simple formule mathématique.

Dans certains cas il faut un modèle d'intelligence artificielle capable d'apprendre les comportements normaux d'un équipement et de détecter les écarts. Le choix du modèle dépend directement de ce que l'arbre décisionnel a défini comme objectif.

- Si l'objectif est de détecter une anomalie on utilisera des modèles de détection d'anomalies non supervisés comme Isolation Forest ou Auto-encoder.

Un auto-encoder est un type de réseau de neurones qui apprend à reconstruire ses propres données d'entrée. Le modèle est entraîné uniquement sur des données normales. Lorsqu'il reçoit une donnée anormale (ex : début de panne), il la reconstruit mal.

- Si l'objectif est de prédire une défaillance future on se tournera vers des modèles de classification supervisés comme Random Forest ou XGBoost qui nécessitent des données historiques étiquetées avec des occurrences de pannes connues.

XGBoost (Extreme Gradient Boosting) est un algorithme de machine Learning. Il apprend à partir de données historiques étiquetées Exemple : 0 = pas de panne, 1 = panne

- Si l'objectif est d'estimer le temps restant avant une panne, ce qu'on appelle le Remaining Useful Life, on utilisera des modèles de régression ou des modèles de deep learning comme les réseaux LSTM qui sont particulièrement adaptés aux séries temporelles.

Les LSTM (Long Short-Term Memory) sont des réseaux de neurones spécialisés dans les séries temporelles. Ils gardent une mémoire des événements passés. Ils analysent l'évolution dans le temps (capteurs, vibrations, température, etc.)

Pour la préparation des données qui alimentent ces modèles, Python avec Pandas et NumPy est utilisé pour construire les features, c'est-à-dire des variables dérivées des données brutes comme des moyennes glissantes, des variations ou des fréquences d'anomalies sur des fenêtres temporelles, qui permettent de mieux représenter le comportement réel des équipements. Pour les données dynamiques en temps réel, Apache Kafka et Apache Spark Streaming permettent de calculer ces features directement sur les flux continus afin d'alimenter les modèles et de produire des KPI dynamiques mis à jour en continu.

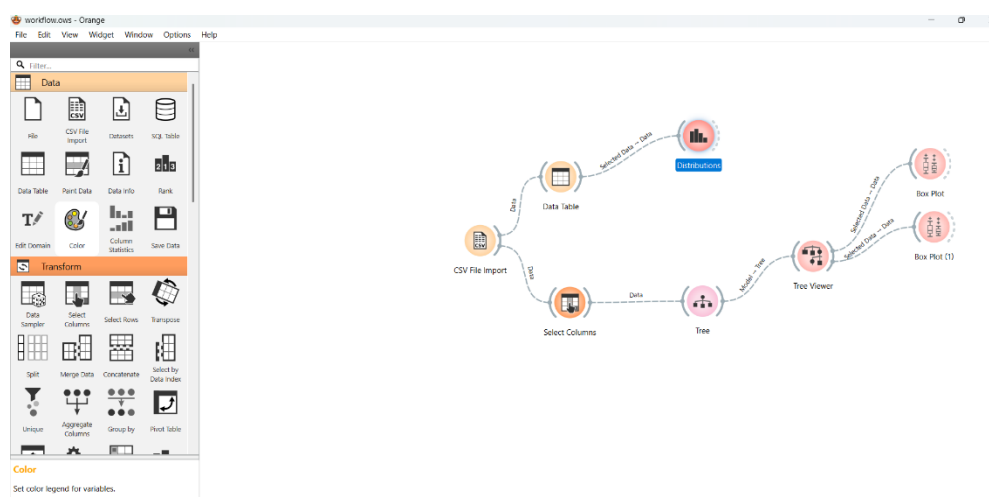
Pour les données dynamiques et en temps réel, la transformation doit prendre en compte le caractère continu des flux. Des outils comme Apache Kafka et Apache Spark Streaming permettent de traiter les données en continu et de calculer des features sur des fenêtres temporelles glissantes, telles que des moyennes mobiles, des variations ou des tendances récentes. Ces informations alimentent ensuite les modèles prédictifs afin de produire en continu des KPI dynamiques, comme une probabilité de panne mise à jour en temps réel ou une estimation du temps restant avant défaillance.



Une fois les modèles entraînés, ceux-ci sont déployés sous forme de services accessibles via des API REST, généralement développées avec FastAPI ou Flask. Ces API permettent d'intégrer les modèles dans une architecture globale où les données des capteurs sont envoyées en temps réel et les résultats des prédictions sont retournés automatiquement.

Les résultats générés par les modèles sont ensuite exploités dans des outils de visualisation sous forme de tableaux de bord interactifs permettant de suivre les KPI, les anomalies détectées et les prédictions de défaillance.

Dans ce processus global, des outils comme **Orange Data Mining** peuvent également être utilisés. Il s'agit d'une plateforme open source de data science permettant de construire visuellement des pipelines complets de traitement de données, couvrant l'ensemble du processus allant de la collecte au prétraitement, à la modélisation et à l'évaluation des modèles de machine learning. Cet outil est particulièrement utile pour prototyper rapidement des modèles sans développement complexe, tout en validant les approches avant leur déploiement dans une architecture industrielle.



Il est également important de noter que l'ensemble de ces étapes peut être géré directement dans Power BI, qui permet de se connecter à des bases de données relationnelles, des fichiers Excel, des API REST ou des flux en temps réel, d'y appliquer des transformations via son moteur Power Query et de construire les KPI directement avec le langage DAX. Pour les données dynamiques, Power BI propose des datasets en streaming qui permettent de recevoir des données en temps réel et de les afficher dans des tableaux de bord qui se rafraîchissent automatiquement, ce qui en fait une plateforme unifiée capable de couvrir l'ensemble de la chaîne dans des contextes où l'on ne souhaite pas multiplier les outils.

Ainsi, l'ensemble de la chaîne allant de la collecte des données jusqu'à la visualisation des KPI repose sur une architecture intégrée combinant data engineering, intelligence artificielle et outils de visualisation, permettant de mettre en place un système complet de maintenance prédictive.



Cas Pratiques : Brûleur de chaudière dans une savonnerie

Nous allons prendre notre exemple réaliste de panne d'un brûleur de chaudière dans une savonnerie, afin d'illustrer l'ensemble de la chaîne allant de la collecte des données jusqu'à la construction des KPI.

- Production : 5 tonnes de savon / jour
- Valeur de production : 500 000 FCFA / jour
- Coût horaire d'arrêt : 20 800 FCFA / heure
- Temps moyen de réparation (MTTR) : 10 heures
- Coût main-d'œuvre horaire : 15 000 FCFA / heure
- Coût pièces : 100 000 FCFA

Etape 1 : Arbre décisionnel

Avant la création des KPI, un arbre décisionnel est construit afin de guider toute la démarche :

Quelle est la cible ? : **brûleur de chaudière**

Quel comportement analyser ? : **dérive thermique et instabilité de flamme**

Quelles données utiliser ? : **température, pression, vibrations, historique des causes arrêts dans la GMAO**

Quelle fréquence de mise à jour ? : **temps réel (flux capteurs) + journalier (GMAO) chaque 6mois**

Quel calcul appliquer ? : **probabilité de panne, RUL, Health Score**

Quelle visualisation utiliser ? : **Dashboard temps réel + graphiques de tendance**

Cet arbre permet de transformer les données brutes en KPI exploitables et directement orientés décision.



Etape 2 : Collecte des données

La collecte repose sur deux sources distinctes qui servent deux objectifs différents. La première source est un dataset historique de cinq ans au format CSV couvrant la période de 2021 à mars 2025 et comprenant la date, la température, la vibration, la stabilité de la flamme et le temps d'arrêt. Un journal de pannes extrait de la GMAO au format CSV complète ces données avec la date des pannes, les types de panne, les causes racines, les actions correctives et les coûts associés. Ces deux jeux de données historiques servent exclusivement à entraîner et évaluer les modèles, en divisant les données en une partie d'entraînement et une partie de test.

	A	B	C	D	E	F	G	H	I	J	K	L
1	date,temperature,pressure,vibration,flame_stability,failure,downtime_hours											
2	2021-01-01	181.4901424590337	4.9723471397675635	0.5323844269050346	0.5238485071795988	0,0						
3	2021-01-02	179.96302868200727	5.337544376277253	0.5468989627908633	1.022925774091268	0,0						
4	2021-01-03	179.82038585127125	4.9502533066627725	0.5325215562197032	1.0945681121821501	0,0						
5	2021-01-04	180.12845959628248	4.8625217305339605	0.54631992058703875	1.0437573681596222	0,0						
6	2021-01-05	186.81630493405902	5.041608276412898	0.5441347093498574	1.069045626848755	0,0						
7	2021-01-06	183.3551675288084	4.878222920039356	0.569649631136267	1.027292208468543	0,0						
8	2021-01-07	181.81912309430817	5.500513524510253	0.5602410064947723	1.0495978752149182	0,0						
9	2021-01-08	180.5621196537976	5.193448652196372	0.4729172820716433	1.0625736690605653	0,0						
10	2021-01-09	187.03678155485194	5.207521074278122	0.5750262278811827	1.0106716231356287	0,0						
11	2021-01-10	183.2606964608376	5.102664239978851	0.643847760490737	0.9778675786722618	0,0						
12	2021-01-11	186.98712635258417	5.139280864632529	0.5665040898454655	0.963067335031621	0,0						
13	2021-01-12	189.4010030223774	5.0699155601748656	0.594502041390134	0.9774094311695478	0,0						
14	2021-01-13	185.793095039357	5.22041107187383	0.564138890978595	1.0532349887382801	0,0						
15	2021-01-14	191.85230951918844	5.266112495311715	0.6789954699742361	0.9747849110763854	0,0						
16	2021-01-15	189.45156860221576	5.809446925924142	0.6382507553619872	0.9140965843325873	0,0						
17	2021-01-16	191.41350545378777	5.340493303168506	0.6321649652908442	0.9871927830910331	0,0						
18	2021-01-17	188.8656523878477	5.415685585024209	0.7299235168315347	1.0171463876260112	0,0						
19	2021-01-18	188.59245231840232	5.548453313477784	0.6812103986984328	1.0171729416952013	0,0						
20	2021-01-19	190.9580897832831	5.580138093132697	0.6382352928040951	1.0065200936312584	0,0						
21	2021-01-20	186.8411788270342	5.466591504262782	0.6847976186562641	0.9893333682637674	0,0						
22	2021-01-21	187.54602708420416	5.344114665868038	0.67275877384181	1.029154959331492	0,0						
23	2021-01-22	193.5600346553814	5.82627091489165	0.706596365398267	0.9756156311487932	0,0						
24	2021-01-23	187.14241825128835	5.464430500140264	0.7086383803804103	0.8647830998551878	0,0						
25	2021-01-24	194.34911263322672	5.483395829626385	0.6547286960331704	0.9302561195482215	0,0						
26	2021-01-25	196.35797795668867	5.328966819602579	0.7906068167892646	1.056941868208107	0,0						
27	2021-01-26	201.09111833692589	5.335140262245369	0.6992176215334871	0.9613188016966316	0,0						
28	2021-01-27	190.3868769553837	5.585256329719442	0.681236026234833	0.9620738031257752	0,0						
29	2021-01-28	200.2218281701255	5.415078161455412	0.7248128052971393	0.9445296185835713	0,0						
30	2021-01-29	196.7713903897315	5.853227297196173	0.6668453036816318	0.9754258413199601	0,0						

	A	B	C	D	E	F	G	H	I
1	date,failure_type,root_cause,corrective_action,downtime_hours,estimated_cost_fcfa,severity								
2	2021-06-25	Flame instability,Air-fuel ratio imbalance,Adjustment of air intake and calibration	8,366400	0,Medium					
3	2023-07-09	Overheating,Sensor malfunction,Replacement of temperature sensor	11,503800	0,Medium					
4	2025-05-28	Flame instability,Air-fuel ratio imbalance,Adjustment of air intake and calibration	10,458000	0,Medium					
5	2023-12-29	Fuel clogging,Impurities in fuel line,Fuel line cleaning and filter replacement	9,412200	0,Medium					
6	2024-01-05	Ignition failure,Electrode wear,Replacement of ignition electrode	8,366400	0,High					
7	2023-12-05	Pressure drop,Pump degradation,Pump replacement and line cleaning	10,458000	0,High					
8	2023-08-08	Ignition failure,Electrode wear,Replacement of ignition electrode	2,91600	0,High					
9	2021-07-21	Overheating,Sensor malfunction,Replacement of temperature sensor	6,274800	0,High					
10	2025-02-16	Pressure drop,Pump degradation,Pump replacement and line cleaning	5,229000	0,Medium					
11	2025-09-25	Flame instability,Air-fuel ratio imbalance,Adjustment of air intake and calibration	9,412200	0,Medium					
12	2023-01-16	Fuel clogging,Impurities in fuel line,Fuel line cleaning and filter replacement	9,412200	0,Medium					
13	2023-01-30	Flame instability,Air-fuel ratio imbalance,Adjustment of air intake and calibration	6,274800	0,Medium					
14	2022-08-03	Pressure drop,Pump degradation,Pump replacement and line cleaning	3,137400	0,Medium					
15	2025-03-21	Pressure drop,Pump degradation,Pump replacement and line cleaning	11,503800	0,High					
16	2023-09-17	Pressure drop,Pump degradation,Pump replacement and line cleaning	4,183200	0,Medium					
17	2022-02-07	Overheating,Sensor malfunction,Replacement of temperature sensor	5,229000	0,Low					
18	2021-02-17	Flame instability,Air-fuel ratio imbalance,Adjustment of air intake and calibration	11,503800	0,High					
19	2022-09-15	Ignition failure,Electrode wear,Replacement of ignition electrode	6,274800	0,Medium					
20	2023-04-08	Pressure drop,Pump degradation,Pump replacement and line cleaning	6,274800	0,Medium					
21	2025-02-07	Fuel clogging,Impurities in fuel line,Fuel line cleaning and filter replacement	9,412200	0,Medium					
22	2021-02-09	Fuel clogging,Impurities in fuel line,Fuel line cleaning and filter replacement	10,458000	0,Low					
23	2022-01-09	Overheating,Sensor malfunction,Replacement of temperature sensor	9,412200	0,Medium					
24	2022-05-11	Pressure drop,Pump degradation,Pump replacement and line cleaning	6,274800	0,High					
25	2021-01-30	Pressure drop,Pump degradation,Pump replacement and line cleaning	8,366400	0,Medium					
26	2022-10-09	Overheating,Sensor malfunction,Replacement of temperature sensor	6,274800	0,Medium					
27	2022-05-02	Pressure drop,Pump degradation,Pump replacement and line cleaning	7,320600	0,Low					
28	2025-01-02	Pressure drop,Pump degradation,Pump replacement and line cleaning	10,458000	0,Medium					
29	2025-09-08	Pressure drop,Pump degradation,Pump replacement and line cleaning	9,412200	0,Low					
30	2022-04-21	Overheating,Sensor malfunction,Replacement of temperature sensor	8,366400	0,Medium					



La deuxième source concerne la collecte en temps réel, qui sert à faire tourner le modèle une fois déployé en production. Dans un environnement industriel complet, cette collecte passerait par un PLC, un SCADA et un Historian.

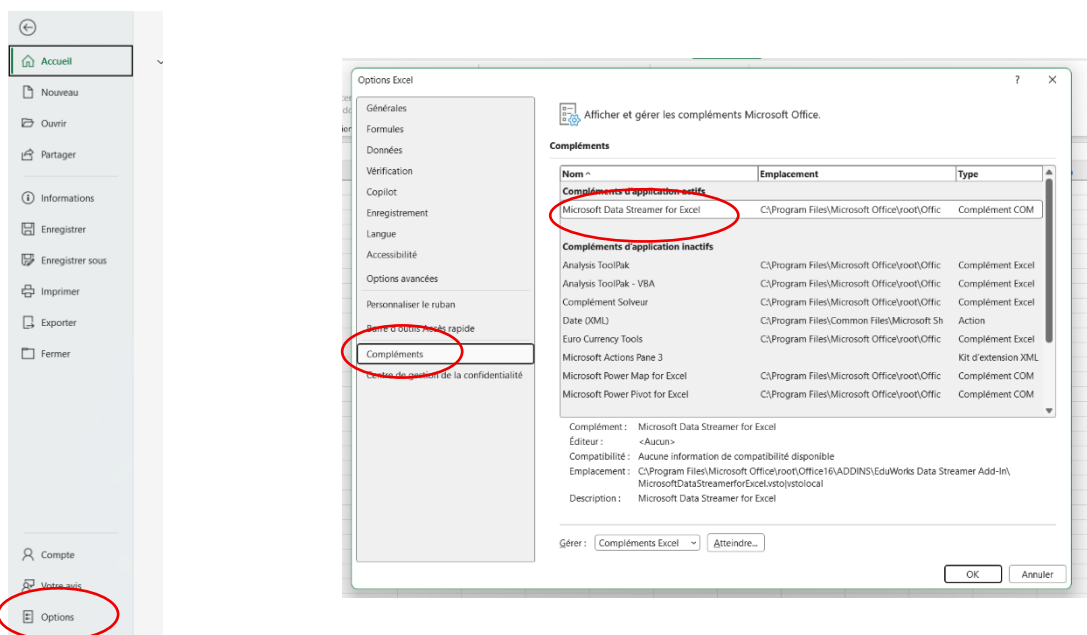
Dans notre exemple pédagogique, nous reproduisons ce principe de manière simplifiée via un module Arduino qui capte les données en continu et les transmet via une liaison série directement dans Excel grâce à l'extension Data Streamer.

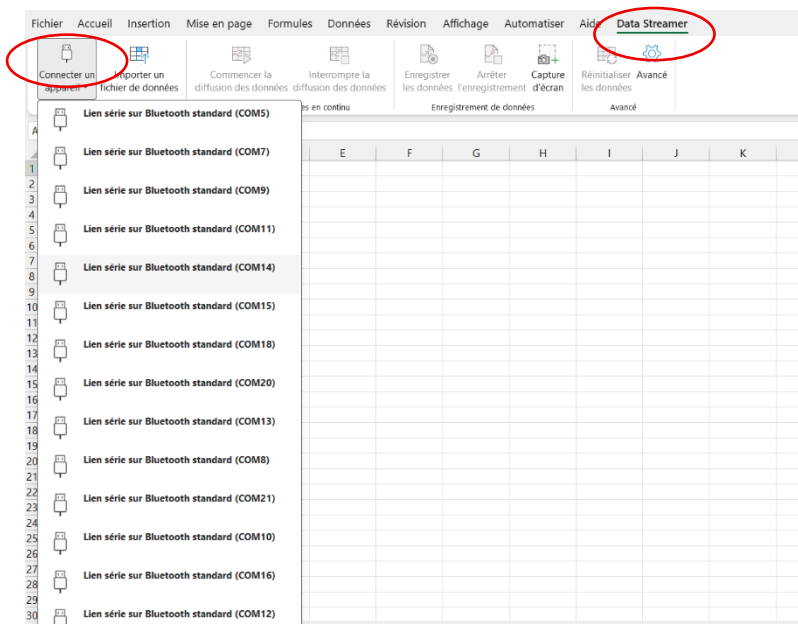
Cette approche reproduit fidèlement le principe de la chaîne industrielle complète dans un environnement accessible. Les données issues de la GMAO, quant à elles, sont mises à jour de manière journalière après chaque intervention, et serviront à réentraîner le modèle tous les six mois afin de maintenir sa fiabilité dans le temps



L'Arduino capte les données en continu et les transmet en temps réel via une liaison série directement dans Excel.

Excel dispose d'une fonctionnalité data streamer qui permet de collecter en temps réel des données ici de capteur





Etape 3 : Traitement des données

Cette étape s'appuie directement sur les éléments définis dans l'arbre décisionnel, notamment les variables à analyser (température, pression, vibration, stabilité de flamme) et les objectifs visés (probabilité de panne, RUL, Health Score).

L'objectif est de fusionner les données capteurs avec les données de panne issues de la GMAO afin de comprendre le comportement du brûleur lors des défaillances.

La fusion de ces deux données nous permettent d'obtenir un seul fichier contenant

- la date
- la température
- la vibration
- la stabilité de la flamme
- le temps d'arrêt
- le type de panne
- la cause racine
- les actions correctives
- le coût associé



Les données sont ensuite traitées à l'aide d'outils comme Python, Pandas et NumPy

Le script ci-dessous présente le code permettant de fusionner et normaliser la variable

```
assets > main.py > ...
1  import pandas as pd
2
3  # Charger les données
4  capteurs = pd.read_csv("bruleur_dynamic_5years.csv")
5  gmao = pd.read_csv("bruleur_history_pannes_5ans.csv")
6
7  # Convertir les dates
8  capteurs['date'] = pd.to_datetime(capteurs['date'])
9  gmao['date'] = pd.to_datetime(gmao['date'])
10
11 # Fusion sur la date (left join pour garder toutes les données capteurs)
12 data = pd.merge(capteurs, gmao, on='date', how='left')
13
14 # Remplacer les valeurs manquantes (pas de panne ce jour-là)
15 data['failure_type'] = data['failure_type'].fillna("Aucune panne")
16 data['downtime_hours_y'] = data['downtime_hours_y'].fillna(0)
17
18 # Renommer pour clarté
19 data.rename(columns={'downtime_hours_x': 'downtime_capteur',
20 | | | | | 'downtime_hours_y': 'downtime_gmao'}, inplace=True)
21
22 # Sauvegarder le dataset fusionné
23 data.to_csv("dataset_fusionne.csv", index=False)
24
25 print("Fusion terminée ✅")
```

A Partir des données brutes, des variables dérivées sont construites afin de mieux représenter le comportement du brûleur : moyennes glissantes de température, variation de pression, niveau de vibration moyen, fréquence des anomalies sur une période donnée, temps écoulé depuis la dernière panne

Ces variables permettent de transformer les données en informations exploitables pour les modèles.

Pour les données dynamiques, le traitement se fait en temps réel via le flux Arduino et Excel Data Streamer afin de produire des indicateurs instantanés.



Étape 4 : Modélisation et entraînement du modèle

Cette étape consiste à entraîner un modèle d'intelligence artificielle en se basant directement sur les questions définies dans l'arbre décisionnel.

Dans notre cas, l'objectif est multiple : détecter une dérive, prédire une panne et estimer le temps restant avant défaillance. Nous allons donc mettre en place plusieurs modèles adaptés à chaque besoin.

Pour la détection de dérive thermique et d'instabilité de flamme, nous allons utiliser un modèle de type Autoencoder.

Ce choix est fait car nous ne disposons pas toujours de données étiquetées pour les anomalies. Le modèle apprendra le fonctionnement normal du brûleur à partir des données capteurs.

Nous utiliserons : la température et ses variations



```
assets > main.py > ...
1 import pandas as pd
2 from sklearn.preprocessing import MinMaxScaler
3 from tensorflow.keras.models import Model
4 from tensorflow.keras.layers import Input, Dense
5
6 # Charger données
7 df = pd.read_csv("dataset_fusionne.csv")
8
9 # Features
10 X = df[['temperature']].values
11
12 # Normalisation
13 scaler = MinMaxScaler()
14 X_scaled = scaler.fit_transform(X)
15
16 # Autoencoder
17 input_dim = X_scaled.shape[1]
18 input_layer = Input(shape=(input_dim,))
19 encoded = Dense(2, activation="relu")(input_layer)
20 decoded = Dense(input_dim, activation="sigmoid")(encoded)
21
22 autoencoder = Model(input_layer, decoded)
23 autoencoder.compile(optimizer='adam', loss='mse')
24
25 # Entraînement
26 autoencoder.fit(X_scaled, X_scaled, epochs=10, batch_size=32)
27
28 print("Autoencoder entraîné")
```

Pour la prédiction de panne, nous allons utiliser un modèle supervisé de type XGBoost. Ce choix est justifié par la disponibilité des données historiques issues de la GMAO, qui permettent de relier les conditions de fonctionnement aux pannes observées.

Les variables utilisées :

- données GMAO (historique de panne)
- variables capteurs

```
assets > main.py > ...
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from xgboost import XGBClassifier
4
5 # Charger données
6 df = pd.read_csv("dataset_fusionne.csv")
7
8 # Création cible (ex : panne ou non)
9 df['failure'] = df['failure_type'].apply(lambda x: 0 if x == "Aucune panne" else 1)
10
11 # Features
12 X = df[['temperature', 'vibration']]
13 y = df['failure']
14
15 # Split
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
17
18 # Modèle
19 model = XGBClassifier()
20 model.fit(X_train, y_train)
21
22 print("Modèle XGBoost entraîné")
```

Pour l'estimation du temps restant avant panne (RUL), nous allons utiliser un modèle basé sur les séries temporelles (LSTM ou régression).

Ce choix est lié au fait que les données capteurs évoluent dans le temps et que la dégradation du brûleur est progressive.

Ici nous utiliserons :

- Temps depuis la dernière panne
- Fréquence des anomalies



assets > main.py > ...

```
1 import pandas as pd
2
3 df = pd.read_csv("dataset_fusionne.csv")
4
5 # Convertir date
6 df['date'] = pd.to_datetime(df['date'])
7
8 # Temps depuis dernière panne
9 df['last_failure_date'] = df['date'].where(df['failure_type'] != "Aucune panne")
10 df['last_failure_date'] = df['last_failure_date'].ffill()
11
12 df['days_since_last_failure'] = (df['date'] - df['last_failure_date']).dt.days
13
14 # Exemple RUL simple (approximation)
15 df['RUL_estime'] = 30 - df['days_since_last_failure']
16
17 print(df[['date', 'RUL_estime']].head())
```

Une fois entraînés, ces modèles produisent les KPI suivants :

KPI 1 : Probabilité de panne

- Il représente le risque de panne à un instant donné
- Il est exprimé en pourcentage

```
df['probabilite_panne'] = model.predict_proba(X)[: ,1]
```

KPI 2 : Score d'anomalie

Ce KPI permet de mesurer le niveau de dérive du système.

- plus le score est élevé plus le comportement est anormal
- utilisé pour détecter les débuts de défaillance

```
import numpy as np

reconstruction = autoencoder.predict(X_scaled)
df['score_anomalie'] = np.mean((X_scaled - reconstruction)**2, axis=1)
```

KPI 3 : RUL (Remaining Useful Life)

Ce KPI représente le temps estimé avant la prochaine panne.

- exprimé en jours



- utilisé pour planifier la maintenance

```
df['RUL'] = 30 - df['days_since_last_failure']
```

KPI 4 : Niveau de criticité

Ce KPI combine plusieurs indicateurs pour prioriser les actions.

- basé sur probabilité de panne + anomalie
- permet de classer les situations

```
df['criticite'] = df['probabilite_panne'] * df['score_anomalie']
```

KPI 5 : État du système (simple pour dashboard)

Permet une lecture rapide pour les opérateurs.

```
def etat(row):  
    if row['probabilite_panne'] > 0.7:  
        return "Critique"  
    elif row['probabilite_panne'] > 0.4:  
        return "Surveillance"  
    else:  
        return "Normal"  
  
df['etat_systeme'] = df.apply(etat, axis=1)
```

MERCI!

EMMANUEL GBRA



Au terme de cette section, retenons que la conception de KPI pour la maintenance prédictive mobilise une chaîne complète d'outils allant de la collecte à la visualisation, mais que tout commence par six questions fondamentales : quelle est la cible, quel comportement analyser, quelles données utiliser, quelle fréquence de mise à jour, quel calcul appliquer et quelle visualisation utiliser. C'est la réponse à ces six questions qui détermine l'ensemble des outils à mobiliser.

Dans notre prochain poste, nous nous intéresserons à l'aspect visuel, c'est-à-dire comment restituer ces données et quel type de visualisation est adapté pour chaque KPI précis.